Verifica scritta di informatica – Classe 4 AIQ – Compito B
Soluzioni (senza `EXISTS`, `ANY`)

Prof. Fedeli Massimo

Soluzioni – Parte A (Query SQL)

1. **Pizze ordinate per categoria e nome**

```
SELECT nome, prezzo, categoria
FROM PIZZE
ORDER BY categoria, nome;
```

2. **Clienti con indirizzo contenente "Via"**

```
SELECT id_cliente, cognome, nome, indirizzo
FROM CLIENTI
WHERE indirizzo LIKE '%Via%';
```

3. **Ordini del cliente con id_cliente = 7**

```
SELECT id_ordine, data_ordine, ora_ordine, tipo,
    stato
FROM ORDINI
WHERE id_cliente = 7;
```

4. **Ordini di febbraio 2026 con dati cliente**

```
SELECT o.id_ordine, c.cognome, c.nome, o.tipo, o.
    stato
FROM ORDINI o
JOIN CLIENTI c ON c.id_cliente = o.id_cliente
WHERE o.data_ordine >= '2026-02-01'
AND o.data_ordine < '2026-03-01';
```

5. **Clienti che non hanno mai effettuato ordini**

```
SELECT c.id_cliente, c.cognome, c.nome
FROM CLIENTI c
LEFT JOIN ORDINI o ON o.id_cliente = c.id_cliente
WHERE o.id_ordine IS NULL;
```

6. **Pizze mai ordinate**

```
SELECT p.id_pizza, p.nome, p.categoria
FROM PIZZE p
LEFT JOIN RIGHE_ORDINE r ON r.id_pizza = p.id_pizza
WHERE r.id_pizza IS NULL;
```

7. **Importo totale per ogni ordine**

```
SELECT id_ordine,
SUM(quantita * prezzo_unitario) AS totale
FROM RIGHE_ORDINE
GROUP BY id_ordine
ORDER BY totale DESC;
```

8. **Ordini senza pagamento con esito positivo**

```
SELECT o.id_ordine, o.id_cliente,
o.data_ordine, o.tipo, o.stato
FROM ORDINI o
LEFT JOIN PAGAMENTI p
ON p.id_ordine = o.id_ordine
AND p.esito = 'OK'
WHERE p.id_pagamento IS NULL;
```

9. **Spesa totale per cliente (solo pagamenti con esito OK)**

```
SELECT c.id_cliente, c.cognome, c.nome,
COALESCE(SUM(p.importo), 0) AS spesa_totale
FROM CLIENTI c
LEFT JOIN ORDINI o  ON o.id_cliente = c.id_cliente
LEFT JOIN PAGAMENTI p ON p.id_ordine = o.id_ordine
AND p.esito = 'OK'
GROUP BY c.id_cliente, c.cognome, c.nome;
```

10. **Clienti che hanno ordinato tutte le pizze della categoria "Classiche"**

    Idea: per ogni cliente, contare le pizze distinte della categoria `'Classiche'` effettivamente ordinate e confrontarle con il totale delle pizze di quella categoria.

```
SELECT c.id_cliente, c.cognome, c.nome
FROM CLIENTI c
JOIN ORDINI o     ON o.id_cliente = c.id_cliente
JOIN RIGHE_ORDINE r ON r.id_ordine = o.id_ordine
JOIN PIZZE p      ON p.id_pizza  = r.id_pizza
WHERE p.categoria = 'Classiche'
GROUP BY c.id_cliente, c.cognome, c.nome
HAVING COUNT(DISTINCT p.id_pizza) = (
SELECT COUNT(id_pizza)
FROM PIZZE
WHERE categoria = 'Classiche'
);
```

Soluzioni – Parte B (Algebra Relazionale)

Nota: si usa l'algebra relazionale estesa con l'operatore di raggruppamento γ.

1. **Nomi delle pizze della categoria "Classiche"**

$$\pi_{\text{nome}}\big(\sigma_{\text{categoria='Classiche'}}(\text{PIZZE})\big)$$

2. **Cognome e nome dei clienti con almeno un ordine**

$$\pi_{\text{cognome, nome}}\big(\text{CLIENTI} \bowtie_{\text{CLIENTI.id_cliente=ORDINI.id_cliente}} \text{ORDINI}\big)$$

3. **Identificativi degli ordini consegnati nel 2026**

$$\pi_{\text{id_ordine}}\big(\sigma_{\text{ stato='consegnato'} \wedge \text{'2026-01-01'} \leq \text{data_ordine} < \text{'2027-01-01'}}(\text{ORDINI})\big)$$

4. **Coppie (cliente, id_ordine) per ordini di tipo "domicilio"**

$$\pi_{\text{CLIENTI.id_cliente, id_ordine}}\big(\text{CLIENTI} \bowtie_{\text{CLIENTI.id_cliente=ORDINI.id_cliente}} \sigma_{\text{tipo='domicilio'}}(\text{ORDINI})\big)$$